

Requirements Analysis

Purpose

The purpose of this document is to use documented requirements from the task plan to make any adjustments to technical and functional requirements necessary to achieve objectives under reasonable constraints as the project evolves. This may include categorizing requirements further into Must Have and Nice to Have, identifying requirement trade-offs through decision schematics, and differentiating between the technical complexity and importance of requirements in requirements decision matrices.

Initial State of Requirements (from Task Plan)

Technical Requirements - Must Have

- a) Python shall be the programming language used for dashboard, API, statistical analysis and modeling development
- b) Cursor shall be used as the ADE (final)
- c) Git and GitHub shall be used for version control (final)
- d) The system shall only accept running activity file types of the .gpx format and fail with an error message if another activity type is input by the user
- e) The system shall display an error message on the dashboard if an invalid numerical value is input for a range for a chart or model/metric/KPI calculation
- f) The system shall automatically detect and handle missing or corrupt fields within a .gpx file, substituting null values and surfacing a warning rather than failing silently
- g) The system shall store all parsed running activity data and model outputs locally in a structured format, avoiding repeated parsing of the same .gpx files on subsequent launches.
- h) All tasks shall run as background processes so the UI remains responsive and interactive while models are being fitted or updated.
- i) The dashboard shall allow for users to upload custom Garmin Activity files of “.gpx” filetype into a repository file path for analysis (final)
- j) The system shall be evaluated using unit testing, integration testing, and user acceptance testing (verification and validation) from end-users (final)
- k) The system shall derive computed fields such as pace, split times, elevation gain/loss, training load from raw .gpx data at ingestion time (final)
- l) The system shall develop time-series models fit to user’s historical data (final)

Technical Requirements - Nice to Have

- a) The system shall generate race time predictions for standard distances (mile, 5K, 10 K, half-marathon, marathon) using a statistical model fitted to the user's historical pace and physiological data.
- b) The system shall display a weekly and monthly training load metric visualised as a chart so the user can identify overtraining or undertraining periods
- c) The system shall identify and flag anomalous runs, sessions where key metrics deviate significantly from the user's personal baseline, with a brief explanatory annotation on the chart.
- d) The system shall project a personalised target race date for a user-specified goal time based on the current field trajectories and historical improvement rates.

Functional Requirements - Must Have

- a) The dashboard shall present all primary metric categories such as biometrics, race predictions, performance trends, and route maps, within a single scrollable view, avoiding navigation to separate pages.
- b) Jira Kanban boards shall be used to track epics, tasks, bugs, and other project elements.
- c) Versions and releases for the MacOSXX shall be documented in GitHub (final).
- d) The dashboard shall be installable as a standalone macOS application (e.g. via a Python-bundled app or a local server launched by a shell script) without requiring the user to interact with a terminal after initial setup (final).
- e) The dashboard shall allow the user to export visualizations, model metrics, indicators, and KPIs as an alternative to automatic data persistence.
- f) The dashboard shall display metrics, indicators, and KPIs (see the task plan for the full list)

Functional Requirements - Nice to Have

- a) All plots shall support interactive zoom and pan so the user can examine fine-grained intervals within a long run or compare multi-week trends.
- b) The dashboard shall include a date-range selector that filters all panels simultaneously, ensuring every metric displayed is consistent with the selected period.

- c) The dashboard shall allow for users to upload custom Garmin Activity files of “.gpx” filetype through drag-and-drop in the dashboard for analysis.
- d) The dashboard shall expose a settings panel allowing the user to configure personal parameters such as resting HR, weight, goal race distance, that are used as inputs to the statistical models.

Requirement Trade-Offs

Overview: In this section, requirements from the task plan that are not final (labelled as “final” on task plan) are analyzed for their trade-offs including strengths and weaknesses, trade-off compromises, and compromise schematics for the purpose of downselecting requirements for system design and a project scope that is more feasible and achievable. The following section will provide an updated list of requirements denoted as “final (needs)”, “later release (wants)”, or “out of scope”.

Technical Requirements - Must Have

1) Single General-Purpose Python Codebase

- **Short Description (Req T1a):** Python shall be the programming language used for dashboard, API, statistical analysis and modeling development
- **Long Description:** Python as single general purpose programming language for codebase enables consolidated workflow with integrated libraries and testing frameworks as well as quick development time (time for developing, testing, debugging, and documenting code), however, with this, we are sacrificing GUI appearance and feature quality as well as responsiveness as Python frontend and dashboard frameworks are limited in quality and responsiveness compared to frontend frameworks like Angular and Vue
- **Core Trade-Offs:** Workflow simplicity, development time, and maintainability vs GUI feature sophistication, GUI appearance quality, and UI responsiveness

- **Trade-Off Summary:**

Strengths

- Increased workflow simplicity from consolidated codebase
- Reduced development time due to unified language and tooling
- Improved maintainability through consistent architecture

Limitations

- Reduced GUI feature sophistication (quality and quantity of features) compared to modern frontend frameworks

- Reduced GUI appearance quality relative to specialized UI frameworks
 - Reduced UI responsiveness under high demands
- **Trade-Off Compromises (How do trade-offs pull on each other?):**
 - **Increasing GUI Feature Sophistication**
 - ↑ Development Time
 - ↓ Workflow Simplicity
 - ↓ Maintainability
 - **Increasing GUI Appearance Quality**
 - ↑ Development Time
 - ↓ Workflow Simplicity
 - **Increasing UI Responsiveness**
 - ↑ Development Time
 - ↓ Maintainability
 - **Increasing Workflow Simplicity**
 - ↓ GUI Feature Sophistication
 - ↓ GUI Appearance Quality
 - ↓ UI Responsiveness
 - **Increasing Maintainability**
 - ↑ Development Time
 - ↓ GUI Feature Sophistication

2) Robust Error Handling and GUI Messaging

- **Short Description (Req T1d - T1f):** The system shall only accept running activity file types of the .gpx format and fail with an error message if another activity type is input by the user. The system shall display an error message on the dashboard if an invalid numerical value is input for a range for a chart or model/metric/KPI calculation. The system shall automatically detect and handle missing or corrupt fields within a .gpx file, substituting null values and surfacing a warning rather than failing silently
- **Long description:** Implementing robust error handling and messages on GUI is great for user experience but results in increased development time, increased memory usage, and possibly reduced application performance (increased runtime latency and/or reduced I/O overhead from additional rendering cycles and API calls).
- **Core Trade-Offs:**
 - User Experience Quality, Application Robustness, and Error Transparency vs Runtime Performance, Memory Usage, and Development Time
- **Trade-Off Summary:**
 - **Strengths**

- Improved User Experience Quality through informative feedback
 - Increased Application Robustness through structured failure handling
 - Increased Error Transparency for debugging and usability
 - Reduced user confusion during failure conditions
- **Limitations**
 - Increased Development Time to design and test messaging workflows
 - Increased Memory Usage due to additional message handling logic
 - Reduced Runtime Performance due to rendering cycles and API calls
- **Trade-Off Compromises:**
 - **Increasing User Experience Quality**
 - ↑ Memory Usage
 - ↑ Development Time
 - ↓ Runtime Performance
 - **Increasing Application Robustness**
 - ↑ Development Time
 - ↑ Memory Usage
 - **Increasing Error Transparency**
 - ↑ Memory Usage
 - ↑ Development Time
 - ↓ Runtime Performance
 - **Increasing Runtime Performance**
 - ↓ Error Transparency
 - ↓ User Experience Quality
 - **Reducing Memory Usage**
 - ↓ Application Robustness
 - ↓ Error Transparency

3) Persisting Parsed Data and Model Outputs

- **Short Description (Req T1g):** The system shall store all parsed running activity data and model outputs locally in a structured format, avoiding repeated parsing of the same .gpx files on subsequent launches.
- **Long description:** Persisting parsed running activity data and model outputs on subsequent launches would increase application performance but require increased development time.
- **Core Trade-Offs:**
 - Runtime Performance and UI Responsiveness vs Development Time, Storage Complexity, Maintainability, and Data Consistency
- **Trade-Off Summary:**

- **Strengths**
 - Improved Runtime Performance by avoiding repeated parsing
 - Improved UI Responsiveness on subsequent launches
 - Reduced redundant computation overhead
 - Faster data retrieval operations
- **Limitations**
 - Increased Development Time to implement persistence mechanisms
 - Increased Storage Complexity due to caching and storage management
 - Reduced Maintainability due to added persistence logic
 - Increased Data Consistency requirements (cache validation)
- **Trade-Off Compromises:**
 - **Increasing Runtime Performance**
 - ↑ Storage Complexity
 - ↑ Development Time
 - ↓ Maintainability
 - **Increasing UI Responsiveness**
 - ↑ Storage Complexity
 - ↑ Data Consistency effort
 - **Increasing Storage Complexity**
 - ↓ Maintainability
 - ↑ Development Time
 - **Increasing Maintainability**
 - ↑ Development Time
 - ↓ Runtime Performance optimization flexibility
 - **Increasing Data Consistency**
 - ↑ Development Time
 - ↓ UI Responsiveness

4) Asynchronous Background Processing

- **Short Description (Req T1h):** All tasks shall run as background processes so the UI remains responsive and interactive while models are being fitted or updated.
- **Long description:** Enabling tasks to run as background processes for UI responsiveness while others are processed concurrently(asynchronous programming) would improve application performance and application UI responsiveness, as it would allow for handling multiple concurrent tasks keeping the UI responsive. However, this comes at the expense of a significant increase in development time (bugs in asynchronous frameworks are much more

challenging to address with race conditions, deadlocks, and timing errors and many data science libraries like pandas and numpy do not benefit from asynchronous programming), and a steep development learning curve as I have limited prior experience with parallel programming.

- **Core Trade-Offs:**
 - UI Responsiveness and Runtime Performance
vs Development Time, Implementation Complexity, Debugging Difficulty, and System Reliability
- **Trade-Off Summary:**
 - **Strengths**
 - Improved UI Responsiveness through non-blocking operations
 - Improved Runtime Performance via concurrent task execution
 - Increased ability to process multiple tasks concurrently
 - Reduced UI blocking during long-running operations
 - **Limitations**
 - Increased Development Time due to concurrency architecture design
 - Increased Implementation Complexity from asynchronous logic and coordination
 - Increased Debugging Difficulty due to race conditions, deadlocks, and timing errors
 - Reduced System Reliability risk tolerance if concurrency issues are not fully mitigated
- **Trade-Off Compromises:**
 - **Increasing UI Responsiveness**
 - ↑ Implementation Complexity
 - ↑ Debugging Difficulty
 - ↑ Development Time
 - ↓ System Reliability (greater concurrency risk exposure)
 - **Increasing Runtime Performance**
 - ↑ Implementation Complexity
 - ↑ Debugging Difficulty
 - ↑ Development Time
 - ↓ System Reliability (greater synchronization risk)
 - **Increasing Implementation Complexity**
 - ↑ Debugging Difficulty
 - ↑ Development Time
 - ↓ System Reliability
 - **Increasing Debugging Difficulty**
 - ↑ Development Time
 - ↓ System Reliability
 - **Increasing System Reliability**
 - ↑ Development Time
 - ↑ Implementation Complexity

- ↓ Runtime Performance

Trade-Off Schematics for Technical Must Have Requirements

Overview: This section takes into account overarching trade-offs, their relationships, and importance to requirements to consider amongst the technical needs/must-have requirements documented in the task plan. These trade-offs are used to down-select requirements as needs vs wants vs out of scope from the initial designations and the schematics are meant to represent the minimum defensible number of trade spaces that fully represent the trade-offs for the technical needs. Trade-Off schematics will be developed in an identical fashion for the technical wants, functional needs, and functional wants sections. After down-selecting requirements from trade-off considerations, the requirements will be documented in requirements decision matrices where they will be grouped by importance (needs vs wants) and difficulty (easy to do vs hard to do) for posterity.

1) Trade Space 1: Primary System Architecture

a) Trade-Offs Pulling Against Each Other:

- i) Development Time
- ii) System Capability (GUI Feature Sophistication, GUI Appearance Quality, UI Responsiveness, Runtime Performance, Application Robustness)
- iii) Maintainability

b) Trade-Off Relationships:

- i) Increasing **System Capability**
 ↑ Development Time
 ↓ Maintainability
- ii) Increasing **Maintainability**
 ↑ Development Time
 ↓ System Capability
- iii) Increasing **Development Time**
 Enables ↑ System Capability
 Enables ↑ Maintainability

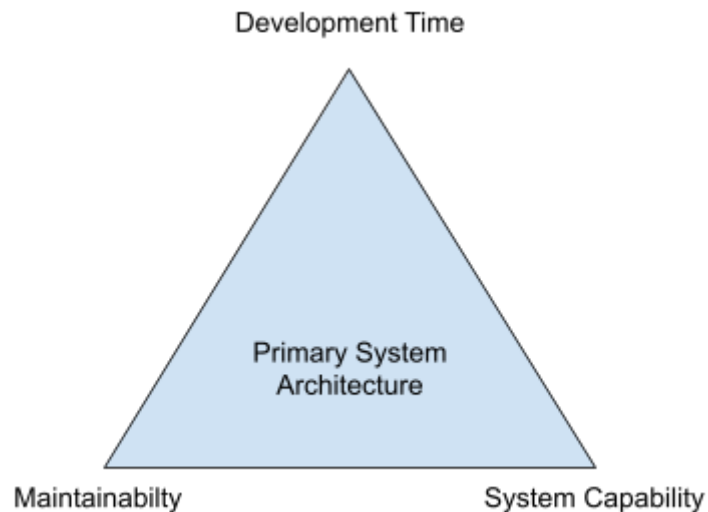
c) Importance of Schematic:

- i) This triangle captures the core tension of engineering capability vs development cost vs long-term sustainability, which appears repeatedly across:
 - (1) Python-only architecture
 - (2) Error handling
 - (3) Data persistence
 - (4) Asynchronous execution

d) Source Requirements Covered:

- i) Python Codebase
- ii) Robust Error Handling

- iii) Persisted Data Storage
- iv) Asynchronous Background Processing



2) Trade Space 2: Execution Stability

a) Trade-Offs Pulling Against Each Other:

- i) Runtime Performance
- ii) UI Responsiveness
- iii) System Reliability
- iv) Debugging Difficulty

b) Trade-Off Relationships

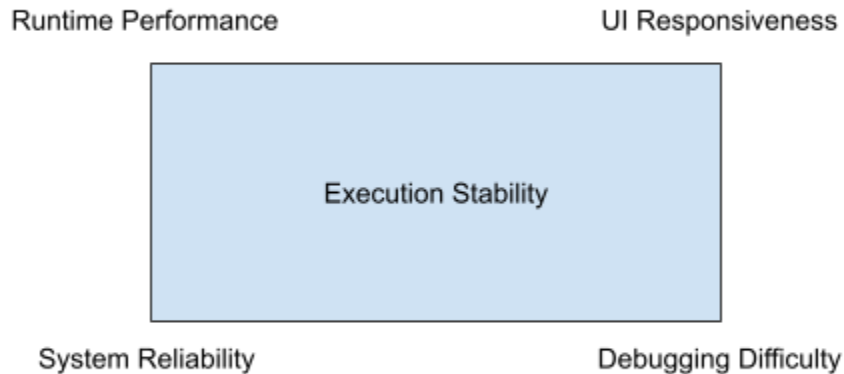
- i) Increasing Runtime Performance
 ↑ Debugging Difficulty
 ↓ System Reliability
- ii) Increasing UI Responsiveness
 ↑ Debugging Difficulty
 ↓ System Reliability
- iii) Increasing Debugging Difficulty
 ↑ Development Effort
 ↓ System Reliability
- iv) Increasing System Reliability
 ↑ Debugging Difficulty
 ↓ Runtime Performance

c) Importance of Schematic:

- i) This diagram isolates the runtime behavior trade space capturing the engineering risk of the system especially critical for:
 - (1) Error Handling
 - (2) Persistence Logic
 - (3) Asynchronous Processing

d) Source Requirements Covered:

- i) Robust Error Handling
- ii) Persisted Data Storage
- iii) Asynchronous Background Processing



3) Trade Space 3: Scalability and Implementation Burden

a) Trade-Offs Pulling on Each Other:

- i) Implementation Complexity
- ii) Memory Usage/Storage Complexity
- iii) Workflow Simplicity

b) Trade-Off Relationships:

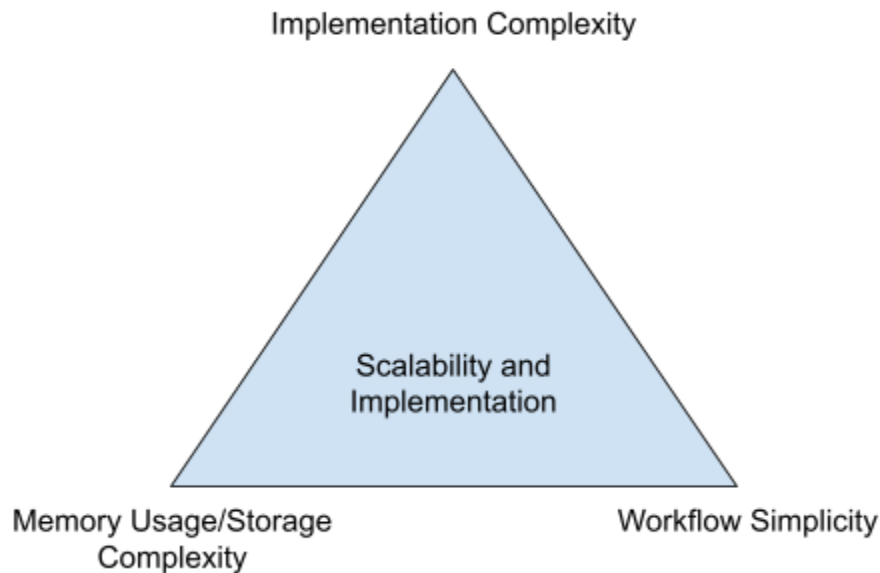
- i) Increasing Implementation Complexity
 - ↑ Storage / Memory Complexity
 - ↓ Workflow Simplicity
- ii) Increasing Storage / Memory Complexity
 - ↓ Workflow Simplicity
 - ↑ Implementation Complexity
- iii) Increasing Workflow Simplicity
 - ↓ Implementation Complexity
 - ↓ Storage / Memory Complexity

c) Importance of Schematic:

- i) This diagram captures the cost of architectural sophistication and engineering maintenance burden and explains why complexity accumulates as features are added covering:
 - (1) Python architecture consolidation
 - (2) Error handling
 - (3) Persistence Logic

d) Source Requirements Covered:

- i) Python Codebase
- ii) Robust Error Handling
- iii) Persisted Data Storage
- iv) Asynchronous Background Processing



Trade-Off Analysis & Downselected Requirements

- a) **Req T1a) Analysis** - The requirement holding that the codebase shall be developed exclusively in Python has trade-offs that span the Primary System Architecture and Scalability and Implementation trade spaces. With regards to the **Primary System Architecture** trade space, the primary trade-offs are system capability (quality and use cases of features), development time, and maintainability. While the dynamic quality and UI responsiveness provided by Python-only frontend and dashboard frameworks may not compare to modern frameworks like Angular and Vue, they make up for it being more purpose-built for data science applications with better integration with Python libraries that will be used for data ingestion, processing, and model development. Furthermore, a Python codebase will significantly reduce development time and maintainability given the task lead's proficiency with the language. The second trade space impacting this requirement is **Scalability and Implementation**. As more languages, frameworks, etc are integrated into a workflow (as workflow simplicity decreases), the implementation complexity and memory usage/storage complexity tend to increase because a more complicated workflow means more interworking software components (implementation complexity) from which increased memory consumption and object/file storage follow. Implementing a Python-only codebase would maximize workflow simplicity, streamlining implementations and making memory and storage management much simpler.
- i) **Decision** - Because a Python codebase will maximize development time and maintainability without sacrificing much system capability and will enable workflow consolidation while keeping implementation and memory complexity low, this requirement is being downselected as **a need**.
- b) **Req T1d Analysis** - The requirement T1d with an error handling scope has implementation trade-offs that span all three trade spaces, **Primary System Architecture, Scalability and Implementation, and Execution Stability**. Regarding

Primary System Architecture, the system capability of the dashboard tool would only be marginally impacted by implementing requirement 1d) that acts as a safeguard to ensure the right file type is input. Users following the application guidelines and knowledgeable of the Garmin ecosystem would know to comply with this requirement. Furthermore, the impact of implementing this requirement on the development time and application maintainability would be light and figures to be straightforward logic. Transitioning to Scalability and Implementation, while trade-offs, there figures to be little impact on the workflow simplicity, implementation complexity, and memory usage beyond what a small block of code would introduce. Lastly, considering Execution Stability, as with development time trade-off, the impact on runtime performance and UI responsiveness as measured by runtime latency, and reduced I/O overhead from rendering cycles and API calls figures to be marginal as do debugging challenges. The impact on system reliability, however, would be more significant since a bug in this error handling logic could prevent any downstream model cleaning, development, or analysis from taking place.

- i) **Decision** - Given the fundamental importance of error handling at the beginning of the application workflow to enable accurate parsing, cleaning, and analysis of historical data, this requirement is being downselected as a **need**. The trade-offs considered in the above trade spaces are not considered significant enough impediments to warrant reconsidering.
- c) **Req T1e Analysis** - The requirement T1e also has implementation trade-offs that span all three trade spaces. Regarding Primary System Architecture considerations, implementing this feature would have a recognizable impact on the system capability as it enables the user to be notified of any range errors when performing a distinct function on the UI, namely, to filter UI chart or data ranges. However, through self-examination, the user should be able to arrive at this conclusion without an accompanying error message. The development time and maintainability could be substantial considering that this would need to be applied to every combination of charts and data values rendered on the UI. In terms of Scalability and Implementation, this implementation figures to have a marginal impact on workflow simplicity and memory usage, but could introduce noticeable implementation complexity. Lastly, considering Execution Stability, as with development time trade-off, the impact on runtime performance and UI responsiveness as measured by runtime latency, and reduced I/O overhead from rendering cycles and API calls figures to be significant as do debugging challenges, and system reliability with test cases needing to span a broad range of charts and analyses.
 - i) **Decision** - Given that this error handling logic does not impact the core functionality of the application which is to parse, clean, model, and visualize historical data, and that some of the trade-offs would mean a noticeable increase in development time, complexity, and impact on system reliability, this requirement is being downselected as a **want** to be considered time permitting.
- d) **Req T1f Analysis** - Requirement T1f has implementation trade-offs that span all three trade spaces. Regarding Primary System Architecture, the system capability of the dashboard tool would only be marginally impacted by implementing requirement 1f) that, similar to requirement T1d, acts as a safeguard to ensure that the input file is not corrupt

and can be successfully processed. The impact of implementing this requirement on the development time and application maintainability would likely be light and figures to be straightforward logic. Transitioning to Scalability and Implementation, while trade-offs, there figures to be little impact on the workflow simplicity, implementation complexity, and memory usage beyond what a small block of code would introduce. Lastly, considering Execution Stability, as with development time trade-off, the impact on runtime performance and UI responsiveness as measured by runtime latency, and reduced I/O overhead from rendering cycles and API calls figures to be marginal as do debugging challenges. The impact on system reliability, however, would be more significant since a bug in this error handling logic could prevent any downstream model cleaning, development, or analysis from taking place.

- i) **Decision** - Given the fundamental importance of error handling at the beginning of the application workflow to enable accurate parsing, cleaning, and analysis of historical data, this requirement is being downselected as a **need** and will be implemented in conjunction with requirement 1d. The trade-offs considered in the above trade spaces are not considered significant enough impediments to warrant reconsidering.
- e) **Req T1g Analysis** - Requirement T1g has trade-offs that span all three trade spaces. Regarding Primary System Architecture, the implementation of this requirement has a significant impact on the system capability of the dashboard tool as persisting parsed data would improve application performance and reduce runtime on future launches. However, this would require an increase in development time that could be substantial requiring conditional logic to check for the existence of identical files between an input folder or drag-and-drop folder from GUI, and only parse new and unique files with many sources of error such as duplicate file names being treated as separate files, incorrect filetypes, etc. The separation of persisted from new data and their organization could get messy and challenging to maintain from a developer standpoint. In terms of Scalability and Implementation trade space, this would have a major impact on the workflow simplicity and introduce implementation complexity because there are multiple downstream actions, processing, analysis, and visualization that could be affected by how the data is persisted requiring the need to move back and forth between data persistence logic and modeling logic. Persisting data would also have a potentially significant impact on memory usage since it would require the application to pull data from multiple file paths. Lastly, regarding Execution Stability, similar to the development time trade-off, the impact on runtime performance could be substantial depending on the number of files that have been persisted. Implementing efficient search algorithms would be required to keep application performance in check. Since the behavior of the persistence feature would need to be tested on downstream functions in the modeling pipeline, this would introduce added debugging complexity. Though the UI responsiveness does not figure to be impacted by this requirement, the impact on system reliability would be significant and could prevent any downstream model cleaning, development, or analysis from taking place or yielding accurate results.
 - i) **Decision** - While the application would benefit from having data persistence logic to improve runtime performance for future launches avoiding duplicative file

processing and traceability, the limitations of increased development time, and maintainability, added testing and debugging complexity, and the possible impact on reliability and accuracy of downstream model results are too significant.

Therefore, this requirement is being down-selected as a **want**.

- f) **Req T1h) Analysis** - This requirement has trade-offs that span the three trade spaces. This requirement has critical application to the system architecture of the application as an asynchronous design would fundamentally change how the tool is developed and tested. Enabling tasks to run as background processes for UI responsiveness while others are processed concurrently (asynchronous programming) would improve system capability, as it would allow for handling multiple concurrent tasks keeping the UI responsive. However, this comes at the expense of a significant increase in development time (bugs in asynchronous frameworks are much more challenging to address with race conditions, deadlocks, and timing errors and many data science libraries like pandas and numpy do not benefit from asynchronous programming), and a steep development learning curve with documentation as I have limited prior experience with parallel programming. In terms of Scalability and Implementation trade space, asynchronous design could introduce a major impediment to workflow simplicity requiring additional logic to be tested in a sequence that differs from synchronous development. Needing to factor race conditions, deadlocks, and timing errors into account introduces significant implementation complexity. Furthermore, while asynchronous development typically increases UI responsiveness allowing multiple requests to be initiated and placed in a queue in quick succession, this often increases peak memory usage especially when the queue length is long, the input sizes are large, or there is processing latency. Lastly, regarding Execution Stability, the runtime performance as indicated by latency and I/O throughput would likely benefit from asynchronous development as would the UI responsiveness through queues. However, the debugging difficulty would be significant given consideration to race conditions, deadlocks, and timing errors not found in sequential/synchronous development. The application would be more reliable from a UI perspective as the user would be able to send data through the UI in quick succession. However, the reliability of data retrieval from the backend and display on the UI would be dependent upon the limitations discussed above that impact memory usage.

- i) **Decision** - The most important trade-off to consider from a development standpoint as this is a concept I'm not very experienced with is the anticipated effect on development time and implementation complexity which appears to be a major factor influenced by nuanced debugging challenges, and different coding logic. From all of the trade spaces, implementation of asynchronous development appears to only benefit the development in a couple of areas, namely, improving UI responsiveness and reliability which will not likely be a pain point should the application be developed in a sequential paradigm given the anticipated use case of small datasets and limited modeling and analysis features for myself and at most, a small number of public users. Therefore, this requirement is being downscoped as **out-of-scope** to not be considered as a need or a want.

Technical Requirements - Nice to Have

1) Fully Personalized Model Development

- **Long Description:** Personalized anomalous running data, targets, models, and KPIs are ideal for long-term race preparation and performance forecasting, as they improve the user experience with tailored insights increasing the use cases, and customization of the dashboard, however, the data cleansing and model development time could be significant since there are many drivers of running performance to consider. Furthermore, this would require the application to build, train, and deploy custom models in real-time degrading application performance and increasing resource utilization. A more efficient alternative may be to leverage personal historical running data possibly coupled with open-source garmin activity data to pre-train general-purpose regression and classification models, reducing runtime demands while still providing meaningful performance insights. It is worth noting that some models, like time-series forecasting models and time-series trends must be fit specifically to the user's custom data.
- **Core Trade-Offs:**
 - User Experience Quality, Use Case Coverage, Model Personalization, and Performance Forecast Accuracy vs Development Time, Runtime Performance, and Resource Utilization
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through tailored insights and anomaly detection
 - Increased Use Case Coverage supporting historical review, race preparation, and long-term planning
 - Increased Model Personalization enabling individualized KPI generation and targets
 - Improved Performance Forecast Accuracy through personalized modeling and historical data utilization
 - Expanded dashboard functionality through customized performance metrics
 - **Limitations**
 - Increased Development Time required for data cleansing, feature engineering, and model development
 - Reduced Runtime Performance when building, training, or deploying models dynamically
 - Increased Resource Utilization from real-time model execution and data processing workloads

- **Trade-Off Compromises:**
 - **Increasing Model Personalization**
 - ↑ Development Time
 - ↑ Resource Utilization
 - ↓ Runtime Performance
 - **Increasing Performance Forecast Accuracy**
 - ↑ Development Time
 - ↑ Resource Utilization
 - ↓ Runtime Performance
 - **Increasing Use Case Coverage**
 - ↑ Development Time
 - ↑ Resource Utilization
 - **Increasing Runtime Performance**
 - ↓ Model Personalization depth
 - ↓ Performance Forecast Accuracy potential
 - **Increasing Resource Utilization**
 - ↓ Runtime Performance
 - ↑ Development Time

Trade-Off Schematics for Technical Nice to Have Requirements

1) Trade Space 1: Model Sophistication

a) Trade-Offs Pulling on Each Other:

- i) Model Personalization
- ii) Performance Forecast Accuracy
- iii) Development Time

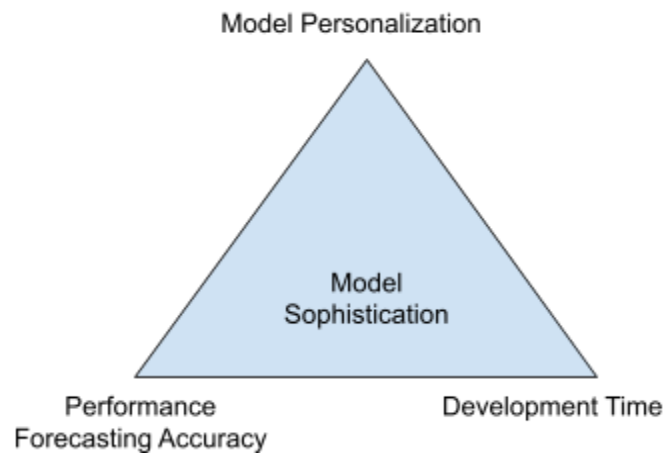
b) Trade-Off Relationships:

- i) Increasing Model Personalization
↑ Development Time
Enables ↑ Performance Forecast Accuracy
- ii) Increasing Performance Forecast Accuracy
↑ Development Time
Requires ↑ Model Personalization
- iii) Increasing Development Time
Enables ↑ Model Personalization
Enables ↑ Performance Forecast Accuracy

c) Importance of Schematic:

- i) This is a core model personalization trade space that clearly represents factors such as fully personalized vs generalized models, tailored forecasting and model fidelity against development feasibility. Personalization and forecasting accuracy naturally increase together, but

both require substantial data cleansing, feature engineering, model design, and model tuning.



2) Trade Space 2: Execution Efficiency

a) Trade-Offs Pulling on Each Other:

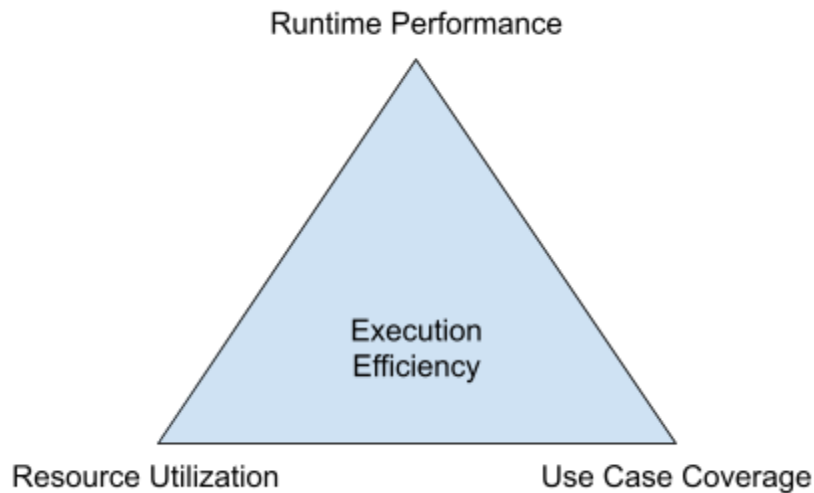
- i) Runtime Performance
- ii) Resource Utilization
- iii) Use Case Coverage

b) Trade-Off Relationships:

- i) Increasing Use Case Coverage
 - ↑ Resource Utilization
 - ↓ Runtime Performance
- ii) Increasing Runtime Performance
 - ↓ Resource Utilization
 - ↓ Use Case Coverage
- iii) Increasing Resource Utilization
 - ↓ Runtime Performance
 - Enables ↑ Use Case Coverage

c) Importance of Schematic:

- i) This schematic distinguishes model quality trade-offs from system execution trade-offs considering runtime impacts from modeling decisions such as personalized models that introduce execution overhead especially when models train dynamically, forecasts update frequently, and multiple model types run concurrently.



Trade-Off Analysis & Downselected Requirements

- a) **Req T2a Analysis** - This requirement has implementation trade-offs that span both the Model Sophistication and Execution Efficiency trade spaces. With regards to the Model Sophistication trade space, increasing model personalization to enable the system to generate tailored race predictions for common distances of 5K, through marathon distance directly increases the performance forecasting accuracy while increasing the development time. While the increase in development time would be substantial, race time predictions for common distances is one of the most desirable forecasting features to implement in a running dashboard making it hard to justify not prioritizing this requirement at the expense of runtime. Considering Execution Efficiency, as the model personalization depth increases so too does the use case coverage of the dashboard metrics and model output for these predictions for training, recovery, nutrition, and other applications. This would have a negative impact on both the runtime performance and the resource utilization, however, since I don't anticipate the throughput or IOPS of the application to be very large based what will likely be models fit to data from files on the order of hundreds in quantity rather than thousands as well as the application being designed to be used by a single user on their local machine, I don't anticipate the runtime performance or resource utilization being a critical sacrifice to enable increased use case coverage for race time predictions.
- i) **Decision** - While implementing support for personalized race time predictions for standard distances comes with trade-off considerations like increased development time, resource utilization, and decreased runtime performance, race time predictions are one of the most beneficial use cases a runner can have from a forecasting model standpoint. Therefore, this requirement is being down-selected as a **need**.

- b) Req T2b Analysis** - This requirement has implementation trade-offs that span both the Model Sophistication and Execution Efficiency trade spaces. With regards to the Model Sophistication trade space, increasing model personalization to enable the system to generate and display weekly and monthly training load metrics for overtraining and undertraining periods is certainly a useful feature to have in a running dashboard especially for high-intensity training periods where the runner is working to establish a PR, attempt a new distance, a new event, is recovering from an injury and needs to be cautious about workload, and other use cases. The development time would be significant as there are likely many drivers of training load that would need to be considered. In terms of Execution Efficiency, enabling this feature would increase the use case of the tool as mentioned above, and would have a similar minor impact on the runtime performance and resource utilization as discussed for Req T2a.
- i) **Decision** - While there are a lot of use cases for a training load metric, it's not as resourceful of a point of reference to gauge running health and future performance as race time forecasts in Req T2a, or physiological indicators and metrics like VO2 max, lactate threshold, and others that are more common in the running community. Therefore, given its low position on the priority list coupled with anticipated increase in development time, this requirement is being down-selected as a **want**.
- c) Req T2c Analysis** - This requirement has implementation trade-offs that span both the Model Sophistication and Execution Efficiency trade spaces. With regards to the Model Sophistication trade space, increasing model personalization to enable the system to identify and flag anomalous runs indicating metrics that deviate from user baselines is certainly a useful feature to have in a running dashboard especially for high-intensity training periods where the runner is working to establish a PR, attempt a new distance, a new event, is recovering from an injury and is very focused on reaching a target goal being aware of any setbacks along the way. The development time would be significant as different metrics would have different criteria for being categorized as anomalous and that would require additional features to evaluate, for each activity, which metrics were deemed anomalous and not which under the hood, requires model fitting to updated datasets. In terms of Execution Efficiency, enabling this feature would increase the use case of the tool as mentioned above, and would have a major impact on the runtime performance and resource utilization as new models and visualization elements would need to be generated.
- i) **Decision** - While there is certainly a use case for anomalous run identification and reporting especially for very targeted training regimens, it's not as resourceful of a point of reference to gauge running health and future performance as race time forecasts in Req T2a, or physiological indicators and metrics like VO2 max, lactate threshold, and others that are more common in the running community. Therefore, given its low position on the priority list coupled with a significant anticipated increase in development time and resource utilization, this requirement is being down-selected as a **want**.
- d) Req T2d Analysis** - This requirement has implementation trade-offs that span both the Model Sophistication and Execution Efficiency trade spaces. With regards to the Model

Sophistication trade space, increasing model personalization to enable the system to project a personalized race date for a user-specified goal is certainly a useful feature to have in a running dashboard especially for newer runners that don't have a target date in mind or could benefit from a trained model projecting one for them based on their condition. The development time would be significant as a personalized target race date would be dependent upon many factors and would require model fitting to updated datasets following each activity. In terms of Execution Efficiency, enabling this feature would increase the use case of the tool as mentioned above, and would have a major impact on the runtime performance and resource utilization as new models would need to be generated.

- i) **Decision** - While there is certainly a use case for generating personalized target race dates especially for newer runners, it's not as resourceful of a point of reference to gauge running health and future performance as race time forecasts in Req T2a, or physiological indicators and metrics like VO2 max, lactate threshold, and others that are more common in the running community. Therefore, given its low position on the priority list coupled with a significant anticipated increase in development time and resource utilization, this requirement is being down-selected as a **want**.

Functional Requirements - Must Have

1) Single-Page Dashboard Layout for Consolidated Navigation

- **Long Description:** All metrics, indicators, KPIs, model results and visualizations should be viewable in a single dashboard page with options to scroll through quickly through widgets and GUI elements to enable efficient and consolidated navigation (ease of navigation) improving the user experience. It's uncertain what the trade-offs of using a single dashboard page will be on the design process but may include development time, application performance, and resource utilization.
- **Core Trade-Offs:**
 - User Experience Quality, Navigation Efficiency, and Workflow Simplicity vs Development Time, Runtime Performance, and Resource Utilization
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through consolidated information display
 - Increased Navigation Efficiency by enabling rapid scrolling through widgets
 - Improved Workflow Simplicity by reducing page transitions
 - Centralized visibility of metrics, KPIs, and model outputs

- Faster interpretation of dashboard insights
- **Limitations**
 - Increased Development Time due to layout coordination and widget integration
 - Reduced Runtime Performance from rendering large volumes of UI components
 - Increased Resource Utilization from maintaining many active visual elements
 - Increased layout complexity affecting long-term maintainability
- **Trade-Off Compromises**
 - **Increasing Navigation Efficiency**
 - ↑ Development Time
 - ↑ Resource Utilization
 - ↓ Runtime Performance
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↑ Resource Utilization
 - **Increasing Workflow Simplicity**
 - ↑ Development Time
 - ↑ Runtime Performance demands
 - **Increasing Runtime Performance**
 - ↓ Navigation Efficiency (fewer simultaneous widgets)
 - ↓ User Experience Quality (reduced visualization density)
 - **Increasing Resource Utilization**
 - ↓ Runtime Performance
 - ↑ Development Time
- **Trade-Off Compromise Schematic:**

2) Kanban-Based Agile Project Management Workflow:

- **Long Description:** Jira Kanban boards are a well-suited option for Agile project management enabling the documentation of progress and any impediments along the way as a continuous flow of work given flexible priorities saving time compared to Agile Scrum where work is planned in fixed sprints which is more heavy of a lift that necessary and not value added for a small one-person development effort. The trade-off of Kanban is less overhead and more flexible work assignments that sacrifice productivity metrics and added structure.
- **Core Trade-Offs:**
 - Workflow Flexibility, Development Time Efficiency, and Workflow Simplicity vs Productivity Measurement Accuracy, Process Structure, and Work Predictability
- **Trade-Off Summary:**

- **Strengths**
 - Improved Workflow Flexibility through dynamic task prioritization
 - Reduced Development Time Efficiency overhead compared to fixed sprint planning
 - Increased Workflow Simplicity suited to solo development
 - Faster response to changing requirements
 - Reduced administrative workload compared to structured sprint frameworks
- **Limitations**
 - Reduced Productivity Measurement Accuracy due to lack of standardized sprint metrics
 - Reduced Process Structure compared to formal sprint planning
 - Reduced Work Predictability without fixed iteration cycles
 - Potential variability in task prioritization discipline
- **Trade-Off Compromises:**
 - **Increasing Workflow Flexibility**
 - ↓ Process Structure
 - ↓ Work Predictability
 - ↓ Productivity Measurement Accuracy
 - **Increasing Workflow Simplicity**
 - ↓ Process Structure
 - ↓ Productivity Measurement Accuracy
 - **Increasing Development Time Efficiency**
 - ↓ Process Structure
 - ↓ Work Predictability
 - **Increasing Process Structure**
 - ↓ Workflow Flexibility
 - ↓ Workflow Simplicity
 - **Increasing Productivity Measurement Accuracy**
 - ↑ Process Structure
 - ↓ Workflow Flexibility
- **Trade-Off Compromise Schematic:**

3) User-Initiated GUI Data Export as Alternative to Automatic Persistence

- **Long Description:** Allowing the user an option through the GUI to export visualizations, model metrics, and other dashboard data from the GUI to their local filesystem offers an alternative to automatic data persistence. With the option to export all dashboard data (ideally as a zip file bundle of multiple filetypes) from the GUI, we are improving the application performance without impacting the development time while also improving the user experience as the user is notified of the export and how to access the persisted data through the interface.
- **Core Trade-Offs:**

- Runtime Performance, User Experience Quality, and Storage Flexibility vs Development Time, Resource Utilization, and Data Accessibility Timing
- **Trade-Off Summary:**
 - **Strengths**
 - Improved Runtime Performance by avoiding constant automatic persistence
 - Improved User Experience Quality through visible export notifications
 - Increased Storage Flexibility allowing users to control data persistence
 - Reduced background storage overhead
 - Simplified persistence strategy through bundled exports
 - Decreased Resource Utilization
 - **Limitations**
 - Increased Development Time to implement export interfaces
 - Reduced Data Accessibility Timing since persistence occurs only when triggered
 - Potential risk of user forgetting to export data manually
- **Trade-Off Compromises:**
 - **Increasing Runtime Performance**
 - ↓ Resource Utilization
 - ↓ Development Time
 - ↓ Data Accessibility Timing
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↑ Resource Utilization
 - **Increasing Storage Flexibility**
 - ↑ Development Time
 - ↑ Resource Utilization
 - **Increasing Resource Utilization**
 - ↓ Runtime Performance
 - ↑ Development Time
 - **Increasing Data Accessibility Timing**
 - ↓ Runtime Performance
 - ↑ Resource Utilization

Trade-Off Schematics for Functional Must Have Requirements

- 1) Trade Space 1: Functional System Behavior
 - a) Trade-Offs Pulling Against Each Other:

- i) User Experience Quality
- ii) Runtime Performance
- iii) Resource Utilization

b) Trade-Off Relationships:

- i) Increasing User Experience Quality
 ↑ Resource Utilization
 ↓ Runtime Performance
- ii) Increasing Runtime Performance
 ↓ Resource Utilization
 ↓ User Experience Quality
- iii) Increasing Resource Utilization
 ↓ Runtime Performance
 Enables ↑ User Experience Quality

c) Importance of Schematic:

- i) This diagram captures the core runtime burden created by user-facing functionality including single-page dashboards, large widgets, and GUI export capabilities. Both the dashboard layout and export workflow produce a UI-driven system load whose functional system behavior is an important trade space.

d) Source Requirements Covered:

- i) Single-Page Dashboard Layout
- ii) User-Initiated GUI Data Export



2) Trade Space 2: Project Management

a) Trade-Offs Pulling Against Each Other:

- i) Workflow Flexibility

- ii) Process Structure
- iii) Work Predictability

b) Trade-Off Relationships:

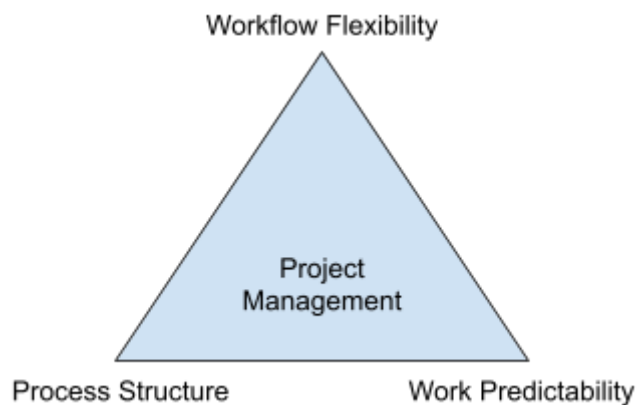
- i) Increasing Workflow Flexibility
 - ↓ Process Structure
 - ↓ Work Predictability
- ii) Increasing Process Structure
 - ↓ Workflow Flexibility
 - Enables ↑ Work Predictability
- iii) Increasing Work Predictability
 - ↑ Process Structure
 - ↓ Workflow Flexibility

c) Importance of Schematic:

- i) This diagram captures the project management and workflow trade space where Kanban trade-offs are organizational, forming a governance loop distinct from functional system behavior.

d) Source Requirements Covered:

- i) Kanban-Based Agile Workflows



Trade-Off Analysis & Downselected Requirements

- a) **Req F1a Analysis** - Enabling a dashboard with a single-page view has trade-offs pertaining to functional system behavior. Implementing a single dashboard page with options to scroll through quickly through widgets and GUI elements to enable efficient and consolidated navigation would improve the user experience permitting quicker insights and a more centralized display. However, there may be significant resource

utilization implications as this would require the need to condense more GUI widgets and other objects into a confined space on the UI affecting the CPU utilization and latency with more interactive components in a fixed view. It would also reduce the runtime performance due to increased widget generation and layout coordination.

- i) **Decision** - While implementing a consolidated display may have negative implications on the runtime performance and resource utilization, these implications are largely speculative and will depend on how the UI objects are developed and rendered. Having a consolidated view that prevents the user from having to perform guesswork and custom navigation to ascertain insights from a large library of UI elements is very important for dashboard user experience. Therefore, this requirement is being down-selected as a **need**.
- b) **Req F1b Analysis** - Enabling an Agile Kanban-based project workflow has a few implications on the project management trade space. Firstly, in comparison to a popular alternative in Agile Scrum, Kanban is a more flexible framework that does not require users to operate in fixed sprint cycles or with unchanging timelines. This Kanban approach is ideal for this effort being a solo development effort as the overhead of the Scrum structure is not necessary. However, with an increase in workflow flexibility comes a reduction in project management process structure as without sprint planning, the documentation is more informal and loosely-coupled. In addition, this comes with a reduction in work predictability as no fixed iteration cycles means a more challenging ability to forecast future work parameters and milestone timelines.
 - i) **Decision** - In conclusion, since an Agile Kanban-based project management workflow enables the flexibility of changing timelines with low overhead, a priority over process structure and measurement, this requirement is being down-selected as a **need**.
- c) **Req F1e Analysis** - Implementing a user-Initiated GUI Data Export feature as an alternative to automatic data persistence has implications on functional system behavior. With the option to export dashboard data (ideally as a zip file bundle of multiple filetypes) from the GUI, we are improving the application performance without impacting the development time granted the user has the option to identify when and which files to export. This also improves the user experience if the user is notified of the export and how to access the persisted data through the interface. In addition, if all data was automatically persisted, not only would this drastically slow application performance after models are fit and dashboard data is rendered for the first time, it would also require the backend to remain in continuous communication with the frontend on persistence through additional logic that determines under what conditions additional data is automatically written out to the local file system. For example, would all of the data be persisted again for each update to the dashboard regardless of how small (ex: updating date range on a chart, changing chart type, other configurations), perhaps only after new models are fit, or there be some set of defined rules that enable persistence. That introduces a lot of complexity in development. With a user-initiated data export capability, we're improving runtime performance and reducing resource utilization by avoiding constant automatic persistence, improving user experience quality and increasing

storage flexibility through visible export notifications and allowing the users to control data persistence only when they want to export data.

- i) **Decision** - Since implementing a user-initiated GUI data export functionality would improve runtime performance, reduce resource utilization and improve user experience, only at the expense of an increase in development time, this requirement is being down-selected as a **need**. Now, I'm uncertain on the status of whether it will be best to incorporate one widget element to enable the user to export all GUI data from a particular state only or to supplement that option with multiple widget elements for the export of individual charts and metrics. That will be decided as the system is designed and implemented.
- d) **Req F1f Analysis** - Generally speaking, this requirement will be implemented as a dashboard's utility is only as good as the metrics, indicators, and especially KPIs it displays. Instead of analyzing for down-selection at the parent level, each documented metric, indicator, and KPI in the task plan will be down-selected as "need", "want", or "out of scope".

- i) **Decision**

- 1) **Metrics (All down-selected as need)**

- (a) Activity Calories: Raw energy expenditure for a specific session
 - (b) Average Heart Rate: The mean beats per minute during an activity.
 - (c) Average Pace: Speed expressed as time per unit of distance.
 - (d) Average Speed: Distance covered divided by time.
 - (e) Average GCT Balance: The symmetry of ground contact between left and right feet.
 - (f) Average Ground Contact Time: The duration your foot is on the ground.
 - (g) Average Run Cadence: Number of steps per minute.
 - (h) Average Stride Length: The physical distance between steps.
 - (i) Average Vertical Oscillation: The "bounce" in your running motion.
 - (j) Average Vertical Ratio: Vertical oscillation divided by stride length.
 - (k) Max Heart Rate: The highest recorded heart rate.
 - (l) Total Activity Time: The raw duration of a workout.
 - (m) Total Distance: The physical length of the activity.
 - (n) BMI: A simple calculation of weight relative to height.

- 2) **Indicators**

- (a) Calories Remaining: Contextualizes calories burned against a daily goal (**need**)
 - (b) HRV Status: Indicates your body's readiness/stress levels based on heart rate variability trends (**want**)
 - (c) Lactate Threshold: Indicates the intensity level where fatigue rapidly increases (**need**)

- (d) Race Predictor: An indicator of potential future performance based on current data **(need)**
- (e) Recovery Advisor: A retrospective indicator suggesting how long to rest after effort **(want)**
- (f) Training Status: Summarizes whether your current load is "Productive," "Peaking," or "Strained." **(want)**
- (g) Workout Recommendation: An actionable suggestion based on recent training trends. **(need)**

3) KPIs

- (a) Endurance Score: A strategic measure of your long-term capacity for sustained effort. **(need)**
- (b) Fitness Age: A high-level interpretation of your overall health relative to your actual age. **(need)**
- (c) Hill Score: A specific KPI for climbing capability and strength. **(want)**
- (d) VO2 Max: The gold-standard KPI for cardiovascular fitness and aerobic power. **(need)**

Functional Requirements - Nice to Have

1) Interactive Visualization Features in GUI

- **Long Description:** Enabling visualizations that support interactive zoom, pan, and interaction features improves user experience increasing use cases and customization of dashboard allowing for drilled-down investigation of trends, however, this may require increased development time, and possibly moderate reduction in application UI responsiveness which may be too significant of a trade-off especially considering that the user can use browser features or desktop image-viewing features to zoom in and out of visualizations.
- **Core Trade-Offs:**
 - User Experience Quality, Use Case Coverage, and Dashboard Customization vs Development Time and UI Responsiveness
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through interactive exploration
 - Increased Use Case Coverage enabling deeper trend investigation
 - Increased Dashboard Customization through flexible viewing interactions
 - Improved analytical capability through drill-down visualization
 - Reduced need for external viewing tools
 - **Limitations**
 - Increased Development Time required to implement interaction logic
 - Reduced UI Responsiveness due to additional rendering demands
 - Increased UI complexity affecting responsiveness stability
 - Potential redundancy with browser-native zoom features
- **Trade-Off Compromises:**
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↓ UI Responsiveness
 - **Increasing Use Case Coverage**
 - ↑ Development Time
 - ↓ UI Responsiveness
 - **Increasing Dashboard Customization**
 - ↑ Development Time
 - ↓ UI Responsiveness
 - **Increasing UI Responsiveness**
 - ↓ Dashboard Customization
 - ↓ User Experience Quality
 - **Increasing Development Time**
 - Enables ↑ Dashboard Customization
 - Enables ↑ Use Case Coverage

2) Global Time-Range Selector for Dashboard Filtering

- **Long Description:** Incorporating a global time-range selector that filters all data in a dashboard view rendered simultaneously improves user experience again, increasing use cases and customization of the dashboard (reducing manual effort for applications where the user Nice to Have all GUI charts, and data to reflect updated time-range). However, the trade-off is that this may result in a marginal increase in development time, marginal decrease in application UI responsiveness, and a marginal reduction in available UI space (amount of unoccupied space on GUI). As a side note, it's not practical from a GUI spatial availability standpoint to consider time-range filters for individual charts, models and other GUI data as it would get too cluttered.
- **Core Trade-Offs:**
 - User Experience Quality, Use Case Coverage, and Dashboard Customization vs Development Time, UI Responsiveness, and Available UI Space
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through synchronized filtering
 - Increased Use Case Coverage supporting rapid time-based analysis
 - Increased Dashboard Customization through flexible time filtering
 - Reduced manual effort for updating multiple visualizations
 - Improved workflow efficiency for temporal comparisons
 - **Limitations**
 - Increased Development Time required to coordinate global filtering logic
 - Reduced UI Responsiveness from re-rendering multiple elements simultaneously
 - Reduced Available UI Space due to selector placement
 - Increased layout coordination requirements
- **Trade-Off Compromises:**
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Dashboard Customization**
 - ↑ Development Time
 - ↓ UI Responsiveness
 - **Increasing Use Case Coverage**
 - ↑ Development Time
 - ↓ UI Responsiveness
 - **Increasing UI Responsiveness**
 - ↓ Dashboard Customization
 - ↓ Use Case Coverage

- **Increasing Available UI Space**
 - ↓ Dashboard Customization
 - ↓ User Experience Quality
- **Trade-Off Compromise Schematic:**

3) Drag-and-Drop Upload for Custom Activity Files

- **Long Description:** Permitting users to upload custom Garmin Activity files through a drag and drop feature is ideal to increase user experience especially for users that are not tech-savvy and may not easily understand how to move custom files into the appropriate file path for ingestion into the system or give them a no-code/browser-based option for data ingestion. However, this implementation may result in marginally-increased development time, marginal to moderate reduction in available UI space or need to render another UI state for data input (another UI state).
- **Core Trade-Offs:**
 - User Experience Quality, Use Case Coverage, and Workflow Simplicity vs Development Time and Available UI Space
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality especially for non-technical users
 - Increased Use Case Coverage through browser-based ingestion
 - Improved Workflow Simplicity by removing manual file-path steps
 - Reduced technical barriers to data ingestion
 - Improved accessibility of system features
 - **Limitations**
 - Increased Development Time required for drag-and-drop logic
 - Reduced Available UI Space due to upload interface components
 - Additional UI state requirements for file input handling
 - Increased layout coordination complexity
- **Trade-Off Compromises:**
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Workflow Simplicity**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Use Case Coverage**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Available UI Space**
 - ↓ Workflow Simplicity
 - ↓ User Experience Quality
 - **Increasing Development Time**

- Enables ↑ Workflow Simplicity
 - Enables ↑ Use Case Coverage
- **Trade-Off Compromise Schematic:**

4) Configurable User Settings Panel for Model Inputs

- **Long Description:** Configuring a settings panel to enable users to configure personal parameters (resting HR, goal weight and race distance, etc.) to be used as inputs to the models would allow for the development of more tailored models (increased user customization) but would likely result in moderate increase in development time, and marginal reduction in available UI space as well as the need to render another UI state.
- **Core Trade-Offs:**
 - User Experience Quality, Model Personalization, and Dashboard Customization vs Development Time and Available UI Space
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through configurable parameters
 - Increased Model Personalization enabling tailored predictions
 - Increased Dashboard Customization through adjustable inputs
 - Improved model relevance through personalized settings
 - Increased adaptability to user-specific goals
 - **Limitations**
 - Increased Development Time required to design configuration interfaces
 - Reduced Available UI Space due to settings panel presence
 - Increased UI state management complexity
 - Additional validation logic requirements
- **Trade-Off Compromises:**
 - **Increasing Model Personalization**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Dashboard Customization**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing User Experience Quality**
 - ↑ Development Time
 - ↓ Available UI Space
 - **Increasing Available UI Space**
 - ↓ Model Personalization
 - ↓ Dashboard Customization
 - **Increasing Development Time**
 - Enables ↑ Model Personalization
 - Enables ↑ Dashboard Customization

Trade-Off Schematics for Functional Nice to Have Requirements

1) Trade Space 1: Feature Capability

a) Trade-Offs Pulling on Each Other:

- i) User Capability
- ii) Development Time
- iii) UI Responsiveness

b) Trade-Off Relationships:

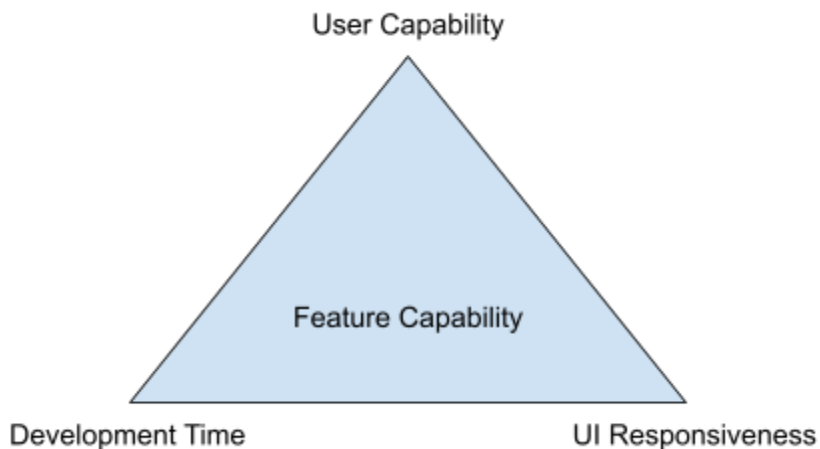
- i) Increasing User Capability
↑ Development Time
↓ UI Responsiveness
- ii) Increasing UI Responsiveness
↓ User Capability
- iii) Increasing Development Time
Enables ↑ User Capability

c) Importance of Schematic:

- i) This diagram covers feature-driven UX growth including user experience quality, use case coverage, and dashboard customization competing with engineering and performance limits. This trade space represents UX sophistication, complexity scaling, and feature latency.

d) Source Requirements Covered:

- i) Interactive Visualization Features
- ii) Global Time-Range Selector
- iii) Drag-and-Drop Upload
- iv) Configurable Settings Panel



2) Trade Space 2: Interface Space

a) Trade-Offs Pulling Against Each Other:

- i) Available UI Space
- ii) Feature Density
- iii) Workflow Simplicity

b) Trade-Off Relationships:

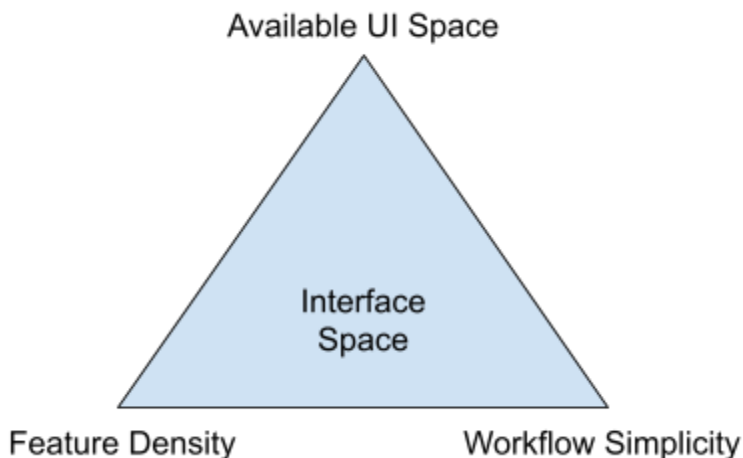
- i) Increasing Feature Density
 - ↓ Available UI Space
 - ↓ Workflow Simplicity
- ii) Increasing Available UI Space
 - ↓ Feature Density
- iii) Increasing Workflow Simplicity
 - ↑ Feature Density
 - ↓ Available UI Space

c) Importance of Schematic:

- i) This diagram characterizes trade-offs in UI availability as features grow capturing the UI layout interface trade space. The time selector, file upload interface, and settings panel widget requirements in this section require UI availability and pull on Trade-Offs like feature density and workflow simplicity.

d) Source Requirements Covered:

- i) Global Time Range Selector
- ii) Drag-and-Drop Upload
- iii) Configurable Settings Panel



Trade-Off Analysis & Downselected Requirements

- a) **Req F2a Analysis** - The requirement that plots on the GUI be interactive has implications on the Feature Capability trade space. Enabling visualizations that support interactive zoom, pan, and other interaction features improves user experience, increasing use cases and customization of the dashboard allowing for drilled-down investigation of trends. However, this may require increased development time, and possibly moderate reduction in application UI responsiveness which may be too significant of a trade-off especially considering that the user can use browser features or desktop image-viewing features to zoom in and out of visualizations.
- i) **Decision** - Since enabling interactivity in the visualizations adds very little benefit to the utility of the dashboard, would decrease UI responsiveness, require additional development time, and can be substituted for native image viewing tools, this requirement is being down-selected as a **want**.
- b) **Req F2b Analysis** - The requirement that the dashboard contain a global date-range selector that filters all data simultaneously ensuring every displayed data element is consistent with the selected period has implications on the Feature Capability and Interface Space trade spaces. With regards to Feature Capability, implementing a feature that allows for dynamic filtering of all model data including charts, metrics, indicators, and KPIs on the GUI could substantially reduce the UI responsiveness as that would require dozens of API calls and rendering cycles for a single updated timeline. This could result in the application freezing (UI becoming unresponsive) depending upon resource utilization. Furthermore, this would require performing additional backend calculations to obtain updated metrics, indicators, and KPIs that reflect performance over the updated timeline. This would add an additional layer of development and testing complexity. Regarding the Interface Space trade space, this additional layer of logic would reduce workflow simplicity as it would require the need to develop another module or class that performs recalculations and updates the rendering of GUI objects. Another thing to consider is that it would have a negative impact on available UI space, and feature density albeit marginal if a single button is used to globally update the date range for all models and associated data.
- i) **Decision** - Implementing a global date-range filter for the GUI adds significant development time and workflow complexity while reducing the UI responsiveness. Having the ability to filter all GUI elements by a specified time range does not provide much of a use case for race preparation or recovery. It's more so a luxury feature. The tool could benefit from the charts displaying somewhere between the last 1-3 months worth of data by default. Therefore, this requirement will be down-selected as a **want**. The default maximum time range for time-series chart displays will be decided in the system design process.
- c) **Req F2c Analysis** - Permitting users to upload custom Garmin Activity files through a drag and drop feature is ideal to increase user experience especially for users that are not tech-savvy and may not easily understand how to move custom files into the appropriate file path for ingestion into the system or give them a no-code/browser-based option for data ingestion. However, this implementation may result in

marginally-increased development time, marginal to moderate reduction in available UI space or need to render another UI state for data input (another UI state). It would also require the user to select whether to drag-and-drop files or permit the application from ingesting from the local file system, another branch in the code also reducing workflow simplicity.

- i) **Decision** - While implementing a UI feature to permit users to upload activity files directly would provide a more user-intuitive use case for data ingestion, it would require increased development time, reduced UI responsiveness, and added workflow complexity. The limitations outweigh the strengths and therefore, this requirement is being down-selected as a **want**. The application will render a warning message to the user that talks to the need to store activity files in the proper directory for ingestion perhaps in the startup window or the first state of the UI in addition to the README file launch procedure.
- d) **Req F2d Analysis** - Implementing a settings panel that enables users to input custom parameter values for model fitting would greatly increase the use cases of the application allowing for forecasts rooted in target physiological data rather than historical data. However, this would require significant development time as logic would need to be incorporated into the backend to fit models to data that supplements or replaces the historical data and to, in turn, update the associated charts, metrics, and other GUI data. This would greatly reduce the UI responsiveness requiring new models to be fit while previous model data remains rendered on the GUI. This would also cause the UI to be much more cluttered, increasing feature density and reducing available UI space.
 - i) **Decision** - While the incorporation of a settings panel would enhance the use cases of the application enabling additional model personalization through custom parameter inputs, the limitations of increased development time, reduced UI responsiveness, and a cluttered UI make this implementation unwise. This requirement is being down-selected as **out-of-scope**.

Downselected Requirements (following Trade-Off Analysis)

Overview: This section provides an updated list of requirements denoted as needs, wants, or out-of-scope.

Technical Requirements

Needs

- 1) Python shall be the primary programming language used in the codebase (down-selected)
- 2) Cursor shall be used as the ADE (final)
- 3) Git and GitHub shall be used for version control (final)
- 4) The system shall only accept running activity file types of the .gpx format and fail with an error message if another activity type is input by the user (down-selected)

- 5) The system shall automatically detect and handle missing or corrupt fields within a .gpx file, substituting null values and surfacing a warning rather than failing silently (down-selected)
- 6) The dashboard shall allow for users to upload custom Garmin Activity files of “.gpx” filetype into a repository file path for analysis (final)
- 7) The system shall be evaluated using unit testing, integration testing, and user acceptance testing (verification and validation) from end-users (final)
- 8) The system shall derive computed fields from raw .gpx data (final)
- 9) The system shall develop time-series models fit to user’s historical data (final)
- 10) The system shall generate race time predictions for standard distances (mile, 5K, 10 K, half-marathon, marathon) using a statistical model fitted to the user's historical pace and physiological data (down-selected).

Wants

- 1) The system shall display an error message on the dashboard if an invalid numerical value is input for a range for a chart or model/metric/KPI calculation (down-selected)
- 2) The system shall store all parsed running activity data and model outputs locally in a structured format, avoiding repeated parsing of the same .gpx files on subsequent launches (down-selected)
- 3) The system shall display a weekly and monthly training load metric visualised as a chart so the user can identify overtraining or undertraining periods (down-selected)
- 4) The system shall identify and flag anomalous runs, sessions where key metrics deviate significantly from the user's personal baseline, with a brief explanatory annotation on the chart (down-selected)
- 5) The system shall project a personalised target race date for a user-specified goal time based on the current field trajectories and historical improvement rates (down-selected)

Out-of-Scope

- 1) All tasks shall run as background processes so the UI remains responsive and interactive while models are being fitted or updated

Functional Requirements

Needs

- 1) The dashboard shall present all model outputs including charts, metrics, indicators, and KPIs within a single scrollable view, avoiding navigation to separate pages (down-selected).
- 2) Jira Kanban boards shall be used to track epics, tasks, bugs, and other project elements (down-selected).
- 3) Versions and releases for the MacOSXX shall be documented in GitHub (final)
- 4) The dashboard shall be launched as a standalone macOS application (e.g. via a Python-bundled app or a local server launched by a shell script) without requiring the user to interact with a terminal after initial setup (final)
- 5) The dashboard shall allow the user to export visualizations, model metrics, indicators, and KPIs as an alternative to automatic data persistence (down-selected)
- 6) The dashboard shall display some combination of the following metrics, indicators, and KPIs (down-selected):

a) Metrics (All down-selected as need)

- i) Activity Calories: Raw energy expenditure for a specific session
- ii) Average Heart Rate: The mean beats per minute during an activity.
- iii) Average Pace: Speed expressed as time per unit of distance.
- iv) Average Speed: Distance covered divided by time.
- v) Average GCT Balance: The symmetry of ground contact between left and right feet.
- vi) Average Ground Contact Time: The duration your foot is on the ground.
- vii) Average Run Cadence: Number of steps per minute.
- viii) Average Stride Length: The physical distance between steps.
- ix) Average Vertical Oscillation: The "bounce" in your running motion.
- x) Average Vertical Ratio: Vertical oscillation divided by stride length.
- xi) Max Heart Rate: The highest recorded heart rate.
- xii) Total Activity Time: The raw duration of a workout.
- xiii) Total Distance: The physical length of the activity.
- xiv) BMI: A simple calculation of weight relative to height.

b) Indicators

- i) Calories Remaining: Contextualizes calories burned against a daily goal
- ii) Lactate Threshold: Indicates the intensity level where fatigue rapidly increases
- iii) Race Predictor: An indicator of potential future performance based on current data

- iv) Workout Recommendation: An actionable suggestion based on recent training trends.

c) KPIs

- i) Endurance Score: A strategic measure of your long-term capacity for sustained effort.
- ii) Fitness Age: A high-level interpretation of your overall health relative to your actual age.
- iii) VO2 Max: The gold-standard KPI for cardiovascular fitness and aerobic power.

Wants

- 1) All plots shall support interactive zoom and pan (down-selected)
- 2) The dashboard shall include a date-range selector that filters all data simultaneously, ensuring every metric displayed is consistent with the selected period (down-selected)
- 3) The dashboard shall allow for users to upload custom Garmin Activity files of ".gpx" filetype through drag-and-drop feature in the UI (down-selected)
- 4) The dashboard shall display some combination of the following metrics, indicators, and KPIs (down-selected):

a) Indicators

- i) HRV Status: Indicates your body's readiness/stress levels based on heart rate variability trends
- ii) Recovery Advisor: A retrospective indicator suggesting how long to rest after effort
- iii) Training Status: Summarizes whether your current load is "Productive," "Peaking," or "Strained."

b) KPIs

- i) Hill Score: A specific KPI for climbing capability and strength.

Out-of-Scope

- 1) The dashboard shall expose a settings panel allowing the user to configure personal parameters such as resting HR, weight, goal race distance, that are used as inputs to the statistical models

Requirements Decision Matrices

Overview: After down-selecting requirements from trade-off considerations, the requirements will be documented in requirements decision matrices where they will be grouped by importance (needs vs wants) and difficulty (easy to do vs hard to do) for posterity.

Technical Requirements

Customer Satisfaction\Technology Differentiation	Easy to Do	Hard to Do
Needs	(Req TN1 - TN3) Tech Stack: Python, Cursor, Git/GitHub (Req TN4 + TN5): Error handling for filetype and fields (Req TN6): Custom file upload into repository path	(Req TN7): Comprehensive system testing (Req TN8): Derive computed fields from .gpx (Req TN9): Personalized time-series forecasting models (generic) (Req TN10): Tailored race time predictions
Wants	(Req TW1): Error handling for invalid UI inputs (Req TW2): Data persistence for future launches	Req TW3): Tailored training load insights (Req TW4): Anomalous run reports (Req TW5): Personalized target race date

Out-of-Scope: All tasks shall run as background processes so the UI remains responsive and interactive while models are being fitted or updated.

Functional Requirements

Customer Satisfaction\Technology Differentiation	Easy to Do	Hard to Do
Needs	(Req FN2): Agile Kanban for PM (Req FN3): GitHub versioning and releases for MacOSXX (Req FN4): Dashboard as standalone application (Req FN6): Calculate and display common metrics	(Req FN1): Single-Page dashboard display (Req FN5): Optional GUI data export (Req FN6): Calculate and display common indicators + KPIs
Wants	(Req FW1): Plot interactivity (Req FW3): Drag-and-drop file upload	(Req FW2): Global date-range selector for UI (Req FW4): Calculate and display uncommon indicators + KPIs

Out-of-Scope: The dashboard shall expose a settings panel allowing the user to configure personal parameters such as resting HR, weight, goal race distance, that are used as inputs to the statistical models